PRÉSENTATION DE STREAMLIT

Streamlit

CRÉER ET PARTAGER DES APPLICATIONS WEB DE MACHINE LEARNING

PAR ANANI DJATO – 08 AVRIL 2026

PLAN

- Définition
- Objectif
- Comment utiliser Streamlit ?
- Créer une application de ML
- Comment déployer une application Streamlit ?
- Conclusion

DÉFINITION

- **Une bibliothèque python gratuite et open-source**
  - Créer des applications web de Machine Learning et de Data Science
  - Partager des applis web

OBJECTIF

- **Accessibilité technique**
  - Pas de connaissance préalable en développement web requise
- **Rapidité de développement**
  - Transformation du code en développement web en un temps record
- **Simplicité de déploiement**
  - Déploiement rapide des modèles ML/Data Science
- **UI efficace**
  - Création d'UI intuitive rapide et modernes
- **Interopérabilité**
  - Compatible avec la majorité des bibliothèques python (Pandas, Matplotlib, Seaborn, Keras, PyTorch...)

OBJECTIF

- **Sobriété du code**
  - Quantité minimale de code => générer des applications performantes
- **Optimisation des performances**
  - Système de mise en cache des données intégré => simplifier et accélérer les pipelines de calcul

UTILISATION DE STREAMLIT

```python
import streamlit as st

st.write("Hello, let's learn how to build a streamlit app together")

st.title("This is the app title")
st.header("This is the header")
st.markdown("This is the markdown")
st.subheader("This is the subheader")
st.caption("This is the caption")
st.code("x = 2026")
st.latex(r''' a+a r^1+a r^2+a r^3 ''')
```

- **Installation sur Linux/Ubuntu**
  - Créer un environnement virtuel : *conda create -n streamlit python=3.13*
  - Installer streamlit : *pip install streamlit*
  - Tester l'installation : *streamlit hello*
  - Exécuter notre code streamlit : *streamlit run file_name.py*
- **Afficher du texte avec Streamlit**
  - streamlit.write() : ajoute du texte à une appli web
  - streamlit.title() : ajoute un titre à l'application
  - streamlit.header() : définie l'en-tête d'une section
  - streamlit.markdown() : définie la démarque d'une section
  - streamlit.sudheader() : ajoute un sous-titre à une section
  - streamlit.caption() : ajoute une légende
  - streamlit.code() : définie du code
  - Streamlit.latex() : affiche des expressions mathématiques formatées en LaTeX

Hello, let's learn how to build a streamlit app together

This is the app title

This is the header

This is the markdown

This is the subheader

This is the caption

```
x = 2026
```

$$a + ar^1 + ar^2 + ar^3$$

UTILISATION DE STREAMLIT

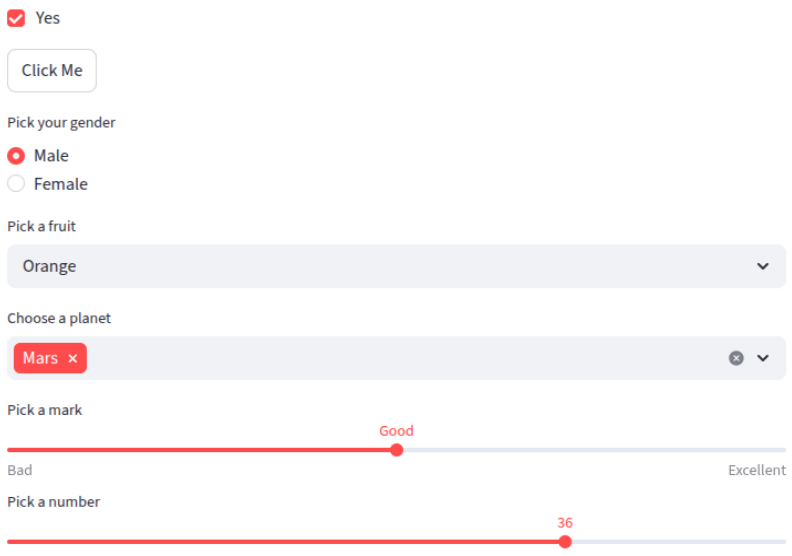
```python
import streamlit as st

st.image("streamlit.png", caption="Logo Streamlit")
```

Logo Streamlit

- **Afficher une image, une vidéo ou un fichier audio avec Streamlit**
  - streamlit.image() : affiche une image
  - streamlit.audio() : affiche un fichier audio
  - streamlit.video() : affiche une vidéo

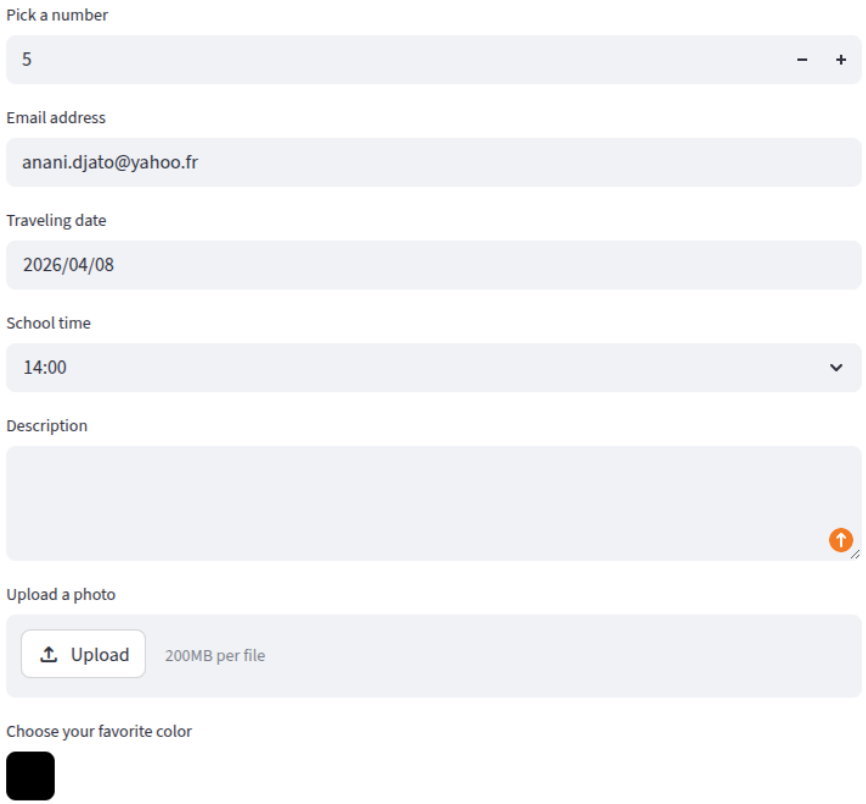
```python
import streamlit as st

st.checkbox('Yes')
st.button('Click Me')
st.radio('Pick your gender', ['Male', 'Female'])
st.selectbox('Pick a fruit', ['Apple', 'Banana', 'Orange'])
st.multiselect('Choose a planet', ['Jupiter', 'Mars', 'Neptune'])
st.select_slider('Pick a mark', ['Bad', 'Good', 'Excellent'])
st.slider('Pick a number', 0, 50)
```

- **Widgets de saisie**
  - streamlit.checkbox() : renvoie True si la case est cochée, False sinon
  - streamlit.button() : affiche un widget de bouton
  - streamlit.radio() : affiche un widget de bouton radio
  - streamlit.selectbox() : affiche un widget sélectionné
  - streamlit.multiselect() : affiche un widget multisélection
  - streamlit.select_slider() : affiche un widget de sélection
  - streamlit.slider() : affiche un widget de type "slider"



UTILISATION DE STREAMLIT

```python
import streamlit as st

st.number_input('Pick a number', 0, 10)
st.text_input('Email address')
st.date_input('Traveling date')
st.time_input('School time')
st.text_area('Description')
st.file_uploader('Upload a photo')
st.color_picker('Choose your favorite color')
```



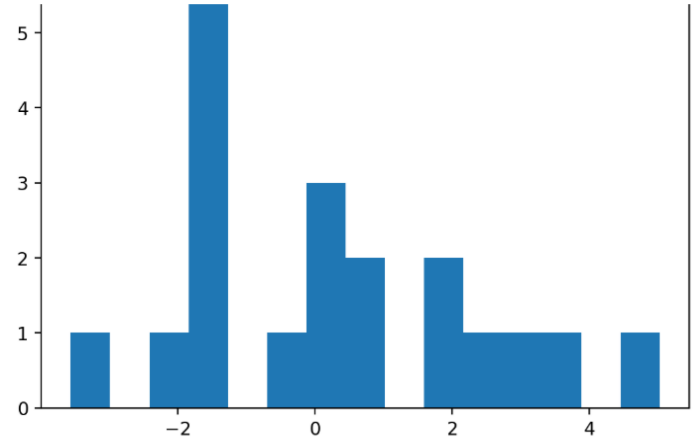
- **Widgets de saisie**
  - streamlit.number_input() : affiche un widget de saisie numérique
  - streamlit.text_input() : affiche un widget de saisie de texte
  - streamlit.date_input() : affiche un widget de saisie de date pour choisir une date
  - Streamlit.time_input() : affiche un widget de saisie de l'heure pour choisir une heure
  - Streamlit.text_area() : affiche un widget de saisie de texte avec plus d'une ligne de texte
  - Streamlit.file_uploader() : affiche un widget de téléchargement de fichiers
  - streamlit.color_picker() : affiche le widget de sélection des couleurs afin de choisir une couleur

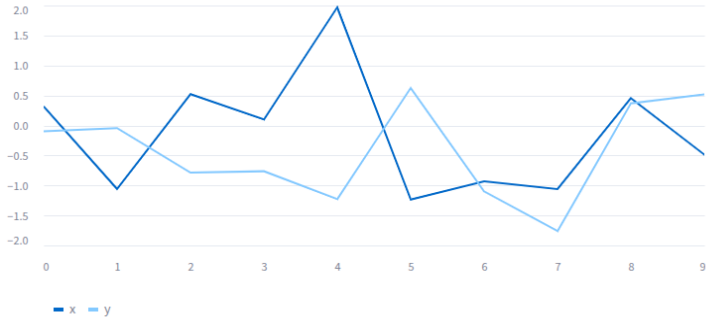
UTILISATION DE STREAMLIT

```python
import streamlit as st
import time

st.balloons() # Celebration balloons

st.subheader("Prgress bar")
st.progress(10)  # Progress barwith

st.subheader("Wait for the execution")
with st.spinner('Wait for it...'):
    time.sleep(10)  # Simulating a process delay


st.success("You did it!")
st.error("Error occurred")
st.warning("This is a warning")
st.info("It's easy to build a Streamlit app")
st.exception(RuntimeError("RuntimeError exception"))
```

- **Afficher les progrès et l'état d'avancement**
  - streamlit.balloons() : affiche des ballons pour une célébration
  - streamlit.progress() : affiche une barre de progression
  - streamlit.spinner() : affiche un message d'attente temporaire pendant l'exécution
  - streamlit.success() : affiche un message de réussite
  - streamlit.error() : affiche un message d'erreur
  - streamlit.warning() : affiche un message d'avertissement
  - streamlit.info() : affiche un message d'information
  - streamlit.exception() : affiche un message d'exception

UTILISATION DE STREAMLIT



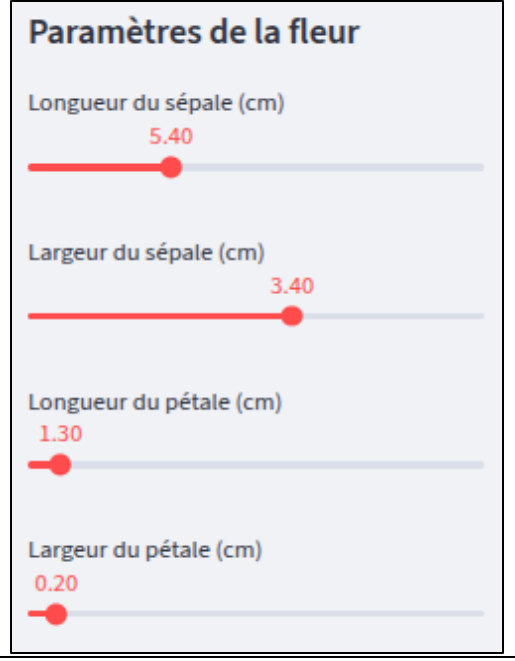


- **Afficher les graphiques**
  - streamlit.pyplot() : affiche une figure matplotlib.pyplot
  - streamlit.line_chart() : affiche un graphique linéaire
  - streamlit.bar_chart() : affiche un diagramme à barres
  - streamlit.area_chart() : affiche un diagramme de zone
  - streamlit.altair_chart() : affiche une carte d'altair
  - streamlit.graphviz_chart(): affiche des objets graphiques, qui peuvent être complétés par différents nœuds et arêtes
  - streamlit.map() : afficher des cartes dans l'application

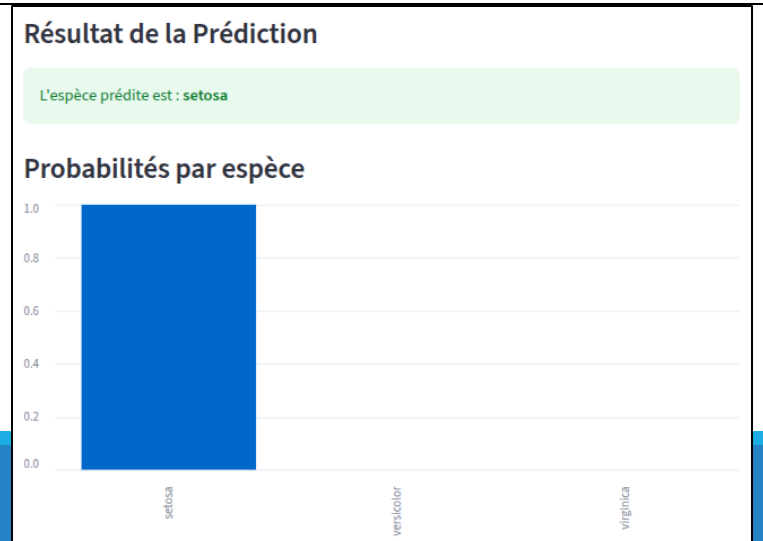
```python
import streamlit as st
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import graphviz

rand = np.random.normal(1, 2, size=20)
fig, ax = plt.subplots()
ax.hist(rand, bins=15)
st.pyplot(fig) # figure matplotlib.pyplot


df = pd.DataFrame(
    np.random.randn(10, 2),
    columns=['x', 'y']
)
st.line_chart(df) # graphique linéaire
```

UTILISATION DE STREAMLIT

- **Créer une application de Machine Learning**
  - Description : Prédire la classe d'un iris parmi 3 espèces (setosa, versicolor, virginica)
  - Dataset : Dataset classique Iris de scikit-learn
  - Modèle ML : Random Forest Classifier
  - Fonctionnalités :
    - **Interface interactive avec Streamlit**
    - **Saisie utilisateur via sliders**
      - Longueur du sépale
      - Largeur du sépale
      - Longueur du pétale
      - Largeur du pétale
    - **Prédiction en temps réel**
    - **Affichage des probabilités par classe**
    - **Modèle entraîné automatiquement (cache optimisé)**



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.4	3.4	1.3	0.2



UTILISATION DE STREAMLIT

- **Créer une application de Machine Learning**
  - Modèle ML :
    - **Algorithme : Random Forest Classifier**
    - **Nombre d'arbres : 100**
    - **Dataset : Iris (sklearn.datasets)**
    - **Features (cm) : sepal length, sepal width, petal length, petal width**

1

```python
import streamlit as st
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

# Configuration de la page
st.set_page_config(page_title="Iris Predictor", page_icon="🌸")

# Titre et introduction
st.title("Classification des Fleurs d'Iris")
st.write("Cette application prédit la variété d'une iris en fonction de ses dimensions.")
```

2

```python
# Chargement des données et entraînement du modèle (mis en cache)
@st.cache_resource
def train_model():
    iris = load_iris()
    X = pd.DataFrame(iris.data, columns=iris.feature_names)
    y = iris.target
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X, y)
    return model, iris.target_names

model, class_names = train_model()

# Barre latérale pour les paramètres de saisie
st.sidebar.header("Paramètres de la fleur")
```

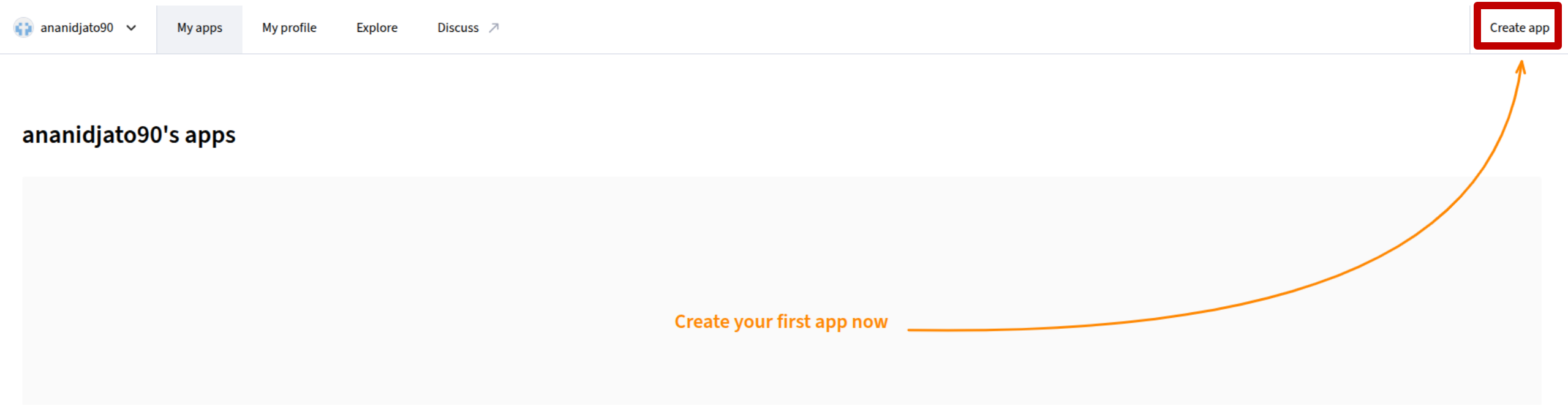
3

```python
def user_input_features():
    sepal_length = st.sidebar.slider('Longueur du sépale (cm)', 4.3, 7.9, 5.4)
    sepal_width = st.sidebar.slider('Largeur du sépale (cm)', 2.0, 4.4, 3.4)
    petal_length = st.sidebar.slider('Longueur du pétale (cm)', 1.0, 6.9, 1.3)
    petal_width = st.sidebar.slider('Largeur du pétale (cm)', 0.1, 2.5, 0.2)

    data = {
        'sepal length (cm)': sepal_length,
        'sepal width (cm)': sepal_width,
        'petal length (cm)': petal_length,
        'petal width (cm)': petal_width
    }
    return pd.DataFrame(data, index=[0])

df_user = user_input_features()

# Affichage des paramètres choisis
st.subheader("Dimensions saisies")
st.write(df_user)

# Prédiction
prediction = model.predict(df_user)
prediction_proba = model.predict_proba(df_user)

# Affichage du résultat
st.subheader("Résultat de la Prédiction")
st.success(f"L'espèce prédite est : **{class_names[prediction[0]]}**")

st.subheader("Probabilités par espèce")
proba_df = pd.DataFrame(prediction_proba, columns=class_names)
st.bar_chart(proba_df.T)
```

DEPLOYER UNE APPLICATION STREAMLIT

- **Comment mettre à disposition de l'application aux utilisateurs ?**
  - Consiste à copier, configurer et activer une application spécifique sur une URL de base spécifique
  - Une fois le processus de déploiement terminé, l'application devient accessible au public sur l'URL de base
  - Le serveur exécute ce processus en deux étapes en mettant d'abord en scène l'application, puis en l'activant une fois la mise en scène réussie
  - Avant de déployer notre application :
    - **Créer un nouveau dépôt sur notre GitHub**
    - **Mettre le code de notre application et ses dépendances**
      - Ajouter le code
      - Créer un nouveau fichier nommé requirements dans lequel placer les bibliothèques utilisées dans notre application
    - Aller sur https://share.streamlit.io/
    - Suivre les étapes suivantes

DEPLOYER UNE APPLICATION STREAMLIT



DEPLOYER UNE APPLICATION STREAMLIT

← Back

What would you like to do?

Deploy a public app from GitHub

My code is ready on a GitHub repo, and it is totally awesome.

Deploy now

Deploy a public app from a template

I want to see what kind of amazing concoctions you have for me.

Check out templates

Deploy a private app in Snowflake

I want unlimited enterprise-grade apps, with the security of Snowflake.

Start trial →

DEPLOYER UNE APPLICATION STREAMLIT

← Back

Deploy an app

Repository

Paste GitHub URL

ananidjato90/streamlit

Branch

main

Main file path

app_ml_iris.py

App URL (optional)

app-iris-class-predictor .streamlit.app

Domain is available

Advanced settings

Deploy

Renseigner les champs puis cliquer sur "Deploy"

DEPLOYER UNE APPLICATION STREAMLIT

Déploiement à finaliser

CONCLUSION

- **Rerun**
  - Relance du script à chaque interaction utilisateur => Application lente si calculs lourds ou API sans mise en cache
- **Personnalisation du design limitée**
  - Mise en page limitée sans contournements techniques () malgré l'existence des colonnes et onglets
    - **Bidouiller avec st.markdown(unsafe_allow_html=True)**
  - Identité visuelle limitée : les apps Streamlit se ressemblent, peu adapté à un produit commercial
- **Scalabilité et performances limitées**
  - Scalabilité limitée : gestion des users simultanés moins efficace, avec une consommation RAM élevée par session
  - Transferts lourds : envoi de gros volumes de données peut fortement ralentir l'interface

CONCLUSION

- **Ecosystème de widget statique**
  - Limité aux composants natifs ou à ceux de la communauté
  - Besoin spécifique = développement en React/TypeScript, perdant l'avantage initial de la simplicité du Python-only
- **Absence de système d'authentification natif**
  - Pas de gestion native des utilisateurs (authentification, rôles, base utilisateurs)
  - Sécurisation via solutions tierces ou offres spécifiques, avec une mise en place complexe

CONCLUSION

- **Vitesse d'exécution rapide**
  - Outil n°1 pour transformer un prototype de Data Science en application interactive en quelques heures plutôt qu'en quelques jours
- **Accessibilité totale**
  - Il démocratise le développement Web pour les profils techniques (Data Scientists, Analystes) en supprimant la barrière du JavaScript/HTML/CSS
- **Idéal pour le prototypage et l'interne**
  - Son terrain de prédilection reste le MVP, les outils de démonstration d'IA et les dashboards opérationnels internes
- **Un compromis "Liberté vs Rapidité"**
  - Le gain de temps se fait au prix d'une personnalisation visuelle plus rigide et d'une gestion de la performance parfois complexe sur des projets de grande envergure
- **Un écosystème en pleine croissance**
  - Malgré ses défauts, la richesse des composants communautaires et l'intégration native avec Snowflake en font un standard incontournable du secteur Data